Phase 1: System Design & Project Roadmap

Before we write a single line of code, we need a blueprint. Every solid engineering project starts with understanding how data flows, how it is stored, and what steps are required to build the final product. Because we are building a full web application, we need to design how the different parts will communicate and plan out our development phases.

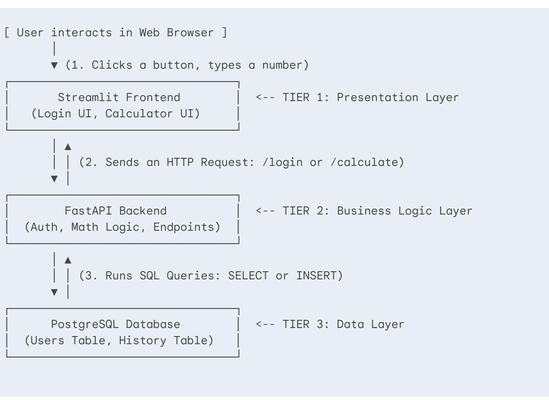
1. System Architecture (The "Big Picture")

We are going to use a classic **3-Tier Architecture**. This means separating our application into three distinct layers. This separation makes the app more secure, easier to update, and easier to scale later.

The Flow of Data:

1. **Tier 1: Presentation (Frontend):** The user interacts with the visual interface in their web browser. We will build this using a Python library called Streamlit.
2. **Tier 2: Logic (Backend API):** The frontend sends instructions (HTTP requests) over the network to our "engine." We will build this using FastAPI.
3. **Tier 3: Data (Database):** The backend processes the logic (like doing the math or checking passwords) and securely saves or retrieves data from PostgreSQL.

Architecture Diagram



Key Concept to Remember: The Python library Streamlit is the frontend UI. It does **not** talk directly to TIER 3: Data Layer (our database). It will send a request to TIER 2: Business Logic Layer (our backend). Then, the backend will check and verify this request. Finally, the backend will talk to the Database and flow the data back.

2. Database Schema (Entity Relationship Diagram)

Now we need to design how we structure the data inside our PostgreSQL database. We need to define tables and how they connect using **Primary Keys (PK)** and **Foreign Keys (FK)**.

For this application, we need a **One-to-Many relationship**: One User can have Many Calculations.

Table 1: `users`

This table stores the registered users. Every time someone signs up, a new row is created here.

Column Name	Data Type	Constraints	Description
`id`	`INTEGER`	**Primary Key**, Auto-increment	The unique identifier for the user (e.g., User 1, User 2).
`username`	`VARCHAR(50)`	Unique, Not Null	The user's chosen login name.
`password_hash`	`VARCHAR(255)`	Not Null	The securely scrambled password. *Never store passwords in plain text!*
`created_at`	`TIMESTAMP`	Default: Current Time	When the account was created.

Table 2: `calculation_history`

This table stores every math problem solved by the calculator. Every time a user clicks "=", a new row is created here.

Column Name	Data Type	Constraints	Description
`id`	`INTEGER`	**Primary Key**, Auto-increment	The unique identifier for this specific calculation.
`user_id`	`INTEGER`	**Foreign Key** (references `users.id`)	Links this calculation to the specific user who made it.
`expression`	`VARCHAR(100)`	Not Null	The math problem (e.g., "5 + 10").
`result`	`FLOAT`	Not Null	The answer (e.g., 15.0).
`created_at`	`TIMESTAMP`	Default: Current Time	Exactly when the calculation was performed.

How They Connect:

Imagine a user named "Alice" registers, and the database assigns her `id = 1` in the `users` table.

Later, Alice logs in and asks the calculator to solve 10×2. The backend will do the math, and then insert a row into the `calculation_history` table where the `user_id` column is set to `1`.

When Alice wants to view her history, the backend will run an SQL query looking for all rows in `calculation_history` where `user_id = 1`.

3. Full Project Roadmap

Now that we understand the architecture and data structure, here is the step-by-step plan for building the complete application.

Phase 2: Building the Backend API (FastAPI)

In this phase, you will write the Python code that handles the business logic and connects to the PostgreSQL database.

- **Set up the Database:** Initialize a local PostgreSQL instance and create the tables defined above.
- **Build the API:** Use FastAPI to create endpoints (`/register`, `/login`, `/calculate`, `/history`).
- **Implement OOP:** Create a `Calculator` class to handle mathematical operations and errors securely.
- **Authentication:** Implement secure password hashing and session management (e.g., using JWTs).

Phase 3: Building the Frontend UI (Streamlit)

Here, you will create the user interface and connect it to your backend.

- **Create Views:** Build screens for Login/Registration and the main Calculator interface.
- **Connect to API:** Write Python code within Streamlit to send HTTP requests to your FastAPI endpoints.
- **Manage State:** Use Streamlit's session state to remember if a user is logged in.
- **Display Data:** Fetch and display the user's calculation history from the backend.

Phase 4: Containerization (Docker)

To ensure the app runs consistently everywhere, you will package it using Docker.

- **Dockerize Frontend:** Create a `Dockerfile` for the Streamlit app.
- **Dockerize Backend:** Create a `Dockerfile` for the FastAPI app.
- **Orchestration:** Write a `docker-compose.yml` file to link the frontend, backend, and a PostgreSQL container together on a single network.

Phase 5: CI/CD & Deployment (GitHub Actions & Azure)

Finally, you will automate the deployment process and make the app live on the internet.

- **Version Control:** Push your code to a GitHub repository.
- **Automate Builds:** Set up GitHub Actions to automatically build your Docker images when you push new code.
- **Cloud Deployment:** Deploy your containers to a cloud service like Azure App Service or Azure Container Apps so anyone can use your calculator.